

# seed42 Audio Module: Task Assignments

COMP3850 PACE, Groups 21 & 22

Phase 1 build. Target: by 30 August, take a few songs played one after another and, every 60 seconds, output a classification we can later turn into a prompt. The analysis must be **causal**: each classification is built from the trailing few seconds of a running feed, never from analysing a whole song in advance. This document says what is already done, the shared shapes everyone builds to, and who does what.

Read **AGENTS.md** in the repo root before writing any code. It has the hard rules (causal only, Phase 1 only, no em dashes). Every task below assumes you have read it.

## 1 What we already have

- Repo created and public. We all work on the **main** branch now (no feature branches): pull the latest, do your task, push back to **main**.
- Project skeleton scaffolded: all folders and stub files, **.gitignore**, **README.md**, **pyproject.toml**, **requirements.txt**.
- **AGENTS.md** coding guidelines, and a setup guide for Anaconda, the Interactive Window, and the git routine.
- The causal streaming loop **io/stream.py**. This is the interface everything else reads from, and everyone's environment is confirmed running it.
- The seed42 Control API spec from the sponsor, so we now know the real output contract (REST calls to **POST /api/prompts**, hot-swap fields only). See Section 4.

So the interface is fixed and the work below runs in parallel now.

## 2 The two shapes everyone builds to

Do not change these without telling Jayden, because other people's code depends on them.

1. A segment, from **AudioStream.segments()**, yields:
  - **timestamp**: a float, seconds, the trailing edge of the window
  - **samples**: a 1D numpy array of the last **window** seconds of audio at sample rate **sr**
2. Every feature function has the same simple signature:

```
def extract(samples, sr) -> dict
```

It takes one segment's samples and the sample rate, and returns a small dict of numbers. No file reading, no state, no future audio. Just numbers out.

3. The shared **MusicState** (Jayden owns this) is one object per window that collects all the feature outputs, with fields roughly like: **timestamp**, **tempo**, **energy**, **brightness**, **flatness**, **key**, **mode**, **valence**, **arousal**. The pipeline fills it each window and the output step reads it.

## 3 Who does what

Each person: work on **main**, read **AGENTS.md**, keep it simple, no em dashes, and commit only your own file.

**Jayden (Ngoc Nhat Pham): the interface and the loop.** Files: `src/seed42_audio/io/stream.py`, `src/seed42_audio/state/music_state.py`, `src/seed42_audio/pipeline.py`. Finish the streaming loop and define the `MusicState` object (a simple dataclass with the fields above). Then wire the pipeline loop that reads segments and emits one classification every 60 seconds. Priority P1, this unblocks everyone.

**Hatim Hussaini: rhythm, plus quality.** File: `src/seed42_audio/features/rhythm.py`. From one segment, extract tempo (BPM), onset positions and beats using `librosa` (`librosa.onset`, `librosa.beat.beat_track`), returned in the `extract(samples, sr)` dict. Also, as Quality Manager, review changes and run the em-dash check (search the source for the em-dash character) before agreeing a piece is done. Priority P1. Level medium to high (tempo is the fiddliest feature).

**Tatiana Denise Bandong: spectral features.** File: `src/seed42_audio/features/spectral.py`. From one segment, compute spectral centroid (brightness), RMS energy (loudness), spectral flatness (noisy vs tonal), and bass energy (low-frequency share). Each is one or two lines in `librosa`. Return them in the `extract(samples, sr)` dict. Priority P1. Level low to medium. Good first task.

**Charminkumar Patel: key and chroma.** File: `src/seed42_audio/features/chroma.py`. From one segment, compute the chroma vector (`librosa.feature.chroma_cqt`), then estimate the key and whether it is major or minor (correlate the chroma against major and minor templates, keep the stronger). Return them in the `extract(samples, sr)` dict. Priority P1. Level low to medium.

**Philopateer Sedrak: mood classification.** File: `src/seed42_audio/mood/classify.py`. Produce valence and arousal (and optionally genre and mood) for each segment, same `extract(samples, sr)` style. **Build the fallback first:** estimate valence and arousal from brightness, energy and mode (the features the others are already producing), so a classification always exists for the 30 August demo. Then *upgrade* to Essentia's pretrained models. Note: plain `essentia` will not install on Windows, use `essentia-tensorflow`, a Mac or Linux machine, or WSL. Doing the fallback first means the demo does not depend on that install. Priority P1. Level medium to high.

**Muhammad Danial Sarfraz: output to seed42 and API docs.** Files: `src/seed42_audio/output/writer.py`, `docs/seed42_api.md`. Two parts: (1) write `docs/seed42_api.md`, a short summary of the Control API (the endpoint, the hot-swap versus reload field list, and our label to field mapping in Section 4); (2) start `writer.py`, the client that sends `POST /api/prompts` with a partial body. Only ever send hot-swap fields. For now it can just print the request it would send, since we do not have a live `stream_id` yet. Priority P2 (not needed for the demo, but good to build now). As liaison, also confirm with the sponsor how we get a `stream_id` and a test stream.

## 4 The seed42 output contract (everyone should know this)

We do not build the renderer. Our module sends a small REST call to seed42's Control API each time something changes, patched onto a running stream. The one rule that matters for everyone:

**Only send hot-swap fields. Never send a reload field.**

Hot-swap fields update the running stream instantly: `prompt`, `seed`, `t_index_list`, `guidance_scale`, `num_inference_steps`, the three `controlnets[].conditioning_scale` values, and `ip_adapter`. Reload fields (`model_id`, `width`, `height`, a ControlNet's model or preprocessor) trigger an approximately 30 second reload with **no output at all**, so they are set once at stream creation and never touched live. Our slider labels map to these fields (worth knowing when we reach the mapping stage): This is for context now and drives the mapping work later. For the 30 August demo we only need the classification output, not the live API call.

| Our label   | Field                                          | Note                                                                  |
|-------------|------------------------------------------------|-----------------------------------------------------------------------|
| Creativity  | <code>t_index_list</code>                      | the UI shows it inverted, so the shown number is not the number sent  |
| Depth       | <code>controlnets[0].conditioning_scale</code> | depth ControlNet, 0 to 1                                              |
| Edge        | <code>controlnets[1].conditioning_scale</code> | canny ControlNet, 0 to 1                                              |
| Consistency | <code>controlnets[2].conditioning_scale</code> | tile ControlNet, feeds the previous output back to hold frames steady |
| Image ref   | <code>ip_adapter</code>                        | style image URL and scale                                             |
| Seed        | <code>seed</code>                              | integer, or weighted tuples                                           |

## 5 How we work (on main, no branches)

Because we all share `main`, always pull before you push, or your push may be rejected. From the project folder in the Anaconda Prompt:

```
git pull                # get everyone's latest first
git status              # check what you changed
git add .               # stage your work (gitignore keeps venv/audio/cache out)
git commit -m "clear message: what and why"
git pull                # pull again in case someone just pushed
git push                # send to main
```

If the push is rejected, run `git pull` then `git push` again. If git reports a conflict (the same lines changed by two people), stop and ask Jayden or Hatim rather than guessing. Because our tasks are in separate files, conflicts should be rare.

## Notes

- Test audio is not in the repo. Put your own short `.wav` in your local `data` folder to test (name it to match `run_stream.py`). On Windows a `.wav` is the most reliable.
- The 30 August critical path: `stream.py` feeds segments, the feature modules and the mood step fill a `MusicState`, and the pipeline prints one classification every 60 seconds. Danial's live API client is the one piece not needed for that demo.
- Take a screenshot of your working output and post it in the group chat. Ask Jayden or Hatim if you are stuck.